

Chapter 1

Introduction to Deep Learning

Giovanni Della Lunga
giovanni.dellalunga@unibo.it

Advanced Topics in Artificial Intelligence

Bologna - March, 2026

Contents

Introduction

About this Lesson

In this lesson, we will cover the following topics:

- **Introduction to Deep Learning.** We will delve into the fundamental building block of deep learning, the neural network. We will cover the architecture of a neural network and its different components.
- **The Feed Forward Architecture.** We will explore the simplest type of neural network, the feed-forward architecture, and learn how it can be used to solve a variety of problems.
- We begin by reviewing core building blocks such as logistic regression, the perceptron model, and gradient-based optimization. Then, we progressively extend these concepts to multi-layer neural networks
- Finally we will introduce the **modern deep learning toolkit**: activation functions (ReLU), optimizers (Adam), regularization techniques, and their application in financial models.

About this Lesson

The focus of the lesson is both theoretical and practical: we aim to provide solid mathematical intuition, as well as hands-on coding experience through the implementation of key algorithms from scratch.

By the end of the lesson, you should be able to:

- Understand how deep learning models are trained and evaluated
- Implement basic architectures such as perceptrons and multilayer networks
- Analyze their performance on synthetic datasets

Artificial Neuron as Linear Classifier

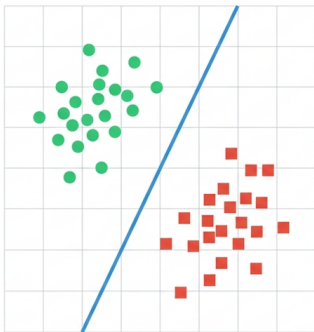
Subsection 1

Linear Separable Set

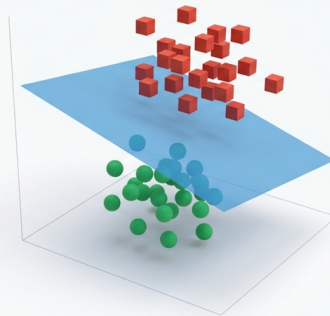
Linear Separation

Linearly Separable Data points: Data points can be said to be linearly separable if a separating boundary/hyperplane can easily be drawn showing distinctively the different class groups.

Hyperplane in $\mathbb{R}^2 \rightarrow$ decision boundary (line)

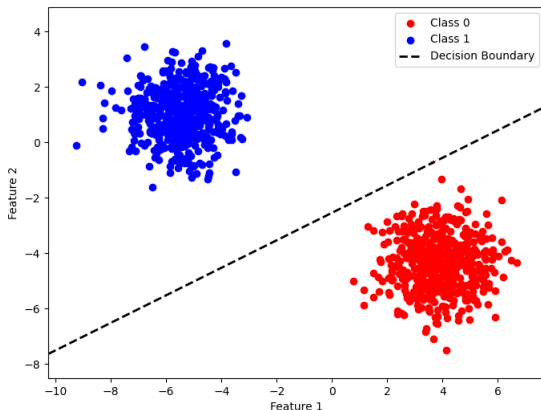


Hyperplane in $\mathbb{R}^3 \rightarrow$ decision boundary (plane)



Problem Definition

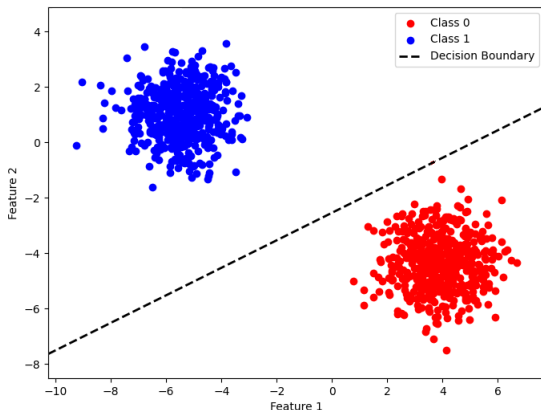
Consider a simple 2D problem where each data point has two features (x_1 and x_2), and we want to classify them into two categories (let's call them **Class 1** and **Class 2**).



Problem Definition

A line can be drawn to separate the data points of **Class 1** from those of **Class 2**.

- **Class 1**: Points that lie on one side of the line (e.g., above the line)
- **Class 2**: Points that lie on the other side of the line (e.g., below the line).



Key Characteristics:

- **2D Case:** In a two-dimensional dataset, the two classes can be separated by a straight line.
- **Line Equation:** The separating line can be defined by the equation:

$$w_1x_1 + w_2x_2 + b = 0$$

Where:

- x_1 and x_2 are the features of the data points.
- w_1 and w_2 are the weights (coefficients) that define the orientation of the line.
- b is the bias, which shifts the line vertically.

Geometrical Interpretation of W

Linear Separation

- The set of weights (w_1, w_2) is nothing more than a vector W .
- But what exactly does W represent?
- We can easily verify that the vector $\mathbf{w} = (w_1, w_2)$ is perpendicular to the line $w_1x_1 + w_2x_2 + b = 0$.
- To demonstrate this relationship we can use the following argument based on the properties of the dot product.

Geometrical Interpretation of W

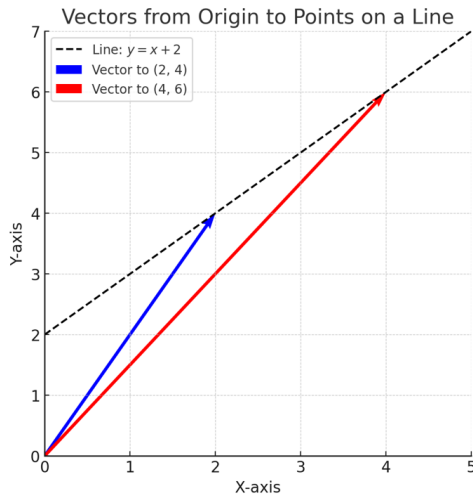
Linear Separation

Step 1: Consider Two Points on the Line

- Let $\mathbf{x} = (x_1, x_2)$ and $\mathbf{y} = (y_1, y_2)$ be any two distinct points on the line.
- Because both points lie on the line, they satisfy the equation:

$$w_1x_1 + w_2x_2 + b = 0$$

$$w_1y_1 + w_2y_2 + b = 0$$



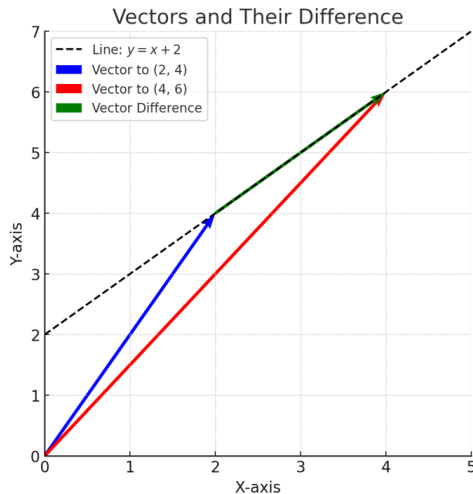
Geometrical Interpretation of W

Linear Separation

Step 2: Form a Tangent Vector to the Line

- A vector that is tangent to the line can be obtained by taking the difference between the two points:

$$\mathbf{v} = \mathbf{y} - \mathbf{x} = (y_1 - x_1, y_2 - x_2).$$



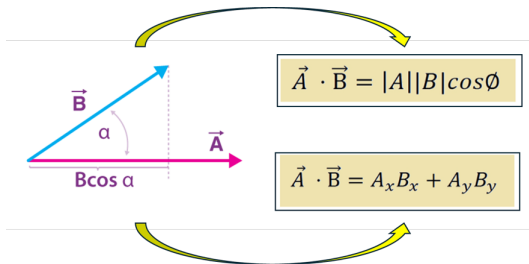
Geometrical Interpretation of W

Linear Separation

Step 3: Show That $\mathbf{w}$ is Perpendicular to $\mathbf{v}$

- To show that $\mathbf{w}$ is perpendicular to the tangent vector $\mathbf{v}$, we need to demonstrate that their dot product is zero:

$$\mathbf{w} \cdot \mathbf{v} = w_1(y_1 - x_1) + w_2(y_2 - x_2).$$



Step 4: Use the Line Equations

- Since both $\mathbf{x}$ and $\mathbf{y}$ lie on the line, we have:

$$w_1x_1 + w_2x_2 + b = 0 \quad \Rightarrow \quad w_1x_1 + w_2x_2 = -b,$$

$$w_1y_1 + w_2y_2 + b = 0 \quad \Rightarrow \quad w_1y_1 + w_2y_2 = -b.$$

- Subtract the first equation from the second:

$$(w_1y_1 + w_2y_2) - (w_1x_1 + w_2x_2) = -b - (-b) = 0.$$

- This simplifies to:

$$w_1(y_1 - x_1) + w_2(y_2 - x_2) = 0 = \vec{w} \cdot (\vec{y} - \vec{x}) \quad \blacksquare$$

Describing Points Above or Below the Line

Linear Separation

Because the vector $\mathbf{w}$ is perpendicular to the line, we can use it to “step off” the line in a controlled way. Specifically, consider any point of the form

$$\mathbf{x} = \mathbf{x}_0 + \alpha \mathbf{u},$$

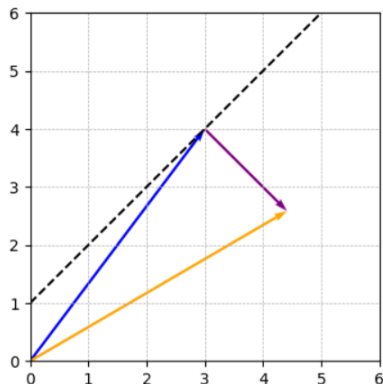
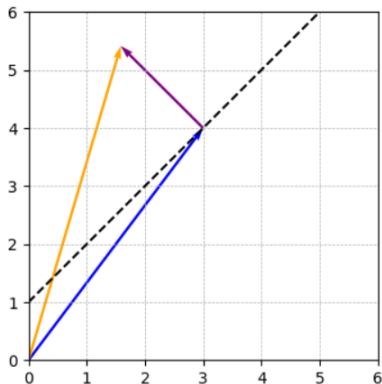
where:

- $\mathbf{x}_0$ is a point on the line (so that $w_1x_{1,0} + w_2x_{2,0} + b = 0$),
- $\mathbf{u}$ is a unit vector in the direction of $\mathbf{w}$, i.e.,

$$\mathbf{u} = \frac{(w_1, w_2)}{\|\mathbf{w}\|},$$

Describing Points Above or Below the Line

Linear Separation



Now, depending on the sign of α , the point $\mathbf{x}$ will lie on one side of the line or the other:

- If $\alpha > 0$, then $\mathbf{x}$ lies in the direction of $\mathbf{w}$ (which we will call “above” the line).
- If $\alpha < 0$, then $\mathbf{x}$ lies in the direction opposite to $\mathbf{w}$ (which we will call “below” the line).

Describing Points Above or Below the Line

Linear Separation

Evaluating the Linear Combination Let's compute the value of

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + b,$$

for the point

$$\mathbf{x} = \mathbf{x}_0 + \alpha \mathbf{u}.$$

Substitute $\mathbf{x}$ into f :

$$\begin{aligned} f(\mathbf{x}) &= w_1(x_{1,0} + \alpha u_1) + w_2(x_{2,0} + \alpha u_2) + b \\ &= (w_1x_{1,0} + w_2x_{2,0} + b) + \alpha(w_1u_1 + w_2u_2). \end{aligned}$$

Describing Points Above or Below the Line

Linear Separation

Since $\mathbf{x}_0$ lies on the line, the first term is zero:

$$w_1 x_{1,0} + w_2 x_{2,0} + b = 0.$$

Now, consider the second term. Because $\mathbf{u}$ is the unit vector in the direction of $\mathbf{w}$:

$$w_1 u_1 + w_2 u_2 = \mathbf{w} \cdot \mathbf{u} = \|\mathbf{w}\| \|\mathbf{u}\| \cos(0) = \|\mathbf{w}\| \cdot 1 \cdot 1 = \|\mathbf{w}\|.$$

Thus, we have:

$$f(\mathbf{x}) = \alpha \|\mathbf{w}\|.$$

Describing Points Above or Below the Line

Linear Separation

Interpreting the Result

1. For Points Above the Line:

When $\alpha > 0$, we get

$$f(\mathbf{x}) = \alpha \|\mathbf{w}\| > 0,$$

because $\|\mathbf{w}\|$ is a positive quantity. Hence, all points that are a positive distance α from the line (in the direction of $\mathbf{w}$) yield

$$w_1x_1 + w_2x_2 + b > 0.$$

Describing Points Above or Below the Line

Linear Separation

Interpreting the Result

2. For Points Below the Line:

When $\alpha < 0$, we obtain

$$f(\mathbf{x}) = \alpha \|\mathbf{w}\| < 0,$$

so all points that are a negative distance α from the line (opposite to the direction of $\mathbf{w}$) yield

$$w_1x_1 + w_2x_2 + b < 0.$$

In the following section we will use three related but distinct objects:

- **Perceptron (Rosenblatt, 1958):** a linear classifier with a *step* activation and a mistake-driven update rule.
- **Logistic Regression:** a *probabilistic* model for binary labels with a Bernoulli likelihood, trained by maximizing likelihood (equivalently minimizing cross-entropy).
- **Sigmoid Neuron:** a differentiable computational unit that becomes a building block for neural networks.

Subsection 2

Perceptron

The Perceptron

- The **Perceptron** is one of the simplest types of artificial neuron and is considered a foundational algorithm in machine learning and neural network theory.
- It was introduced by Frank Rosenblatt in 1958 and is primarily used for binary classification tasks, i.e., determining whether an input belongs to one of two classes.
- A Perceptron works by classifying input vectors through a linear decision boundary, which **is adjusted during the training phase to minimize classification errors**.

The Perceptron

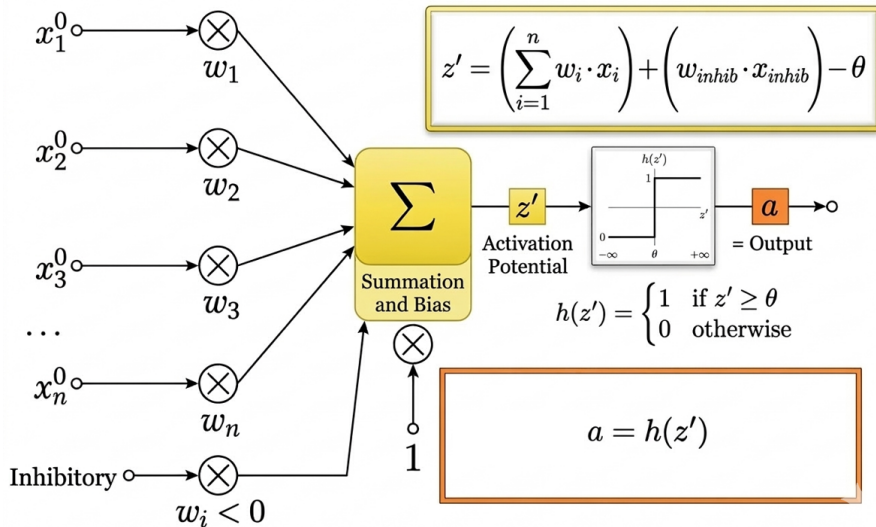
Model:

$$\hat{y} = \mathbb{I}(w^\top x + b \geq 0) \quad \text{with } \hat{y} \in \{0, 1\}.$$

Properties:

- Produces a **hard decision** (not a probability).
- Learns a **linear decision boundary**.
- Can solve **linearly separable** datasets; fails on XOR-like problems.

The Perceptron



Perceptron Learning Rule

Given a labeled example (x_i, y_i) with $y_i \in \{0, 1\}$:

$$\hat{y}_i = \mathbb{I}(w^\top x_i + b \geq 0)$$

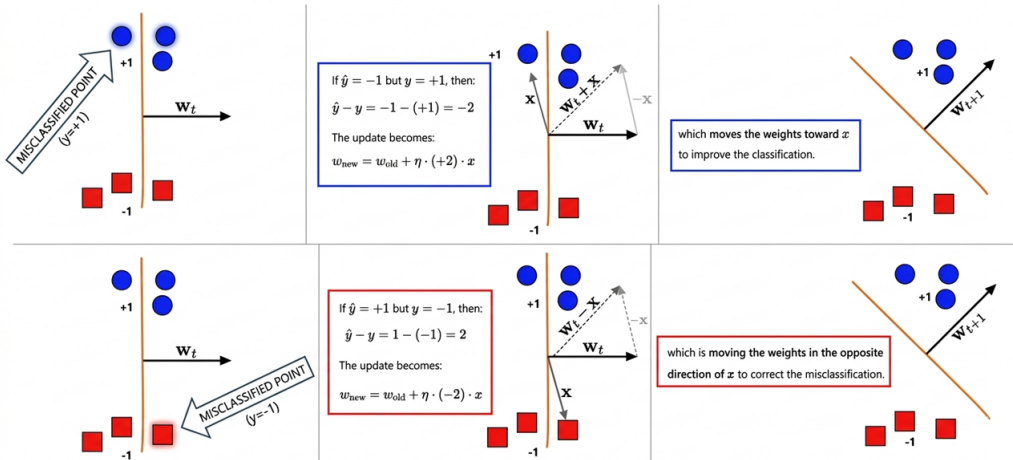
Update (only on mistakes):

$$w \leftarrow w + \eta(y_i - \hat{y}_i)x_i, \quad b \leftarrow b + \eta(y_i - \hat{y}_i).$$

Important:

- This is **not** gradient descent on cross-entropy.
- The step activation is **not differentiable**, so standard backprop-style training does not apply.

The Perceptron



What the Perceptron Is *Not*

The perceptron is not:

- a probabilistic model ($\hat{y}$ is not a probability),
- trained by maximizing likelihood,
- trained by minimizing cross-entropy with gradient descent.

Then why do we still teach it?

- It is historically important and conceptually simple.
- It makes clear the limitation of linear separability.
- It motivates the transition to **differentiable** models.

Subsection 3

Logistic Regression and Sigmoid Neuron

Logistic Regression: A Probabilistic Model

Goal: model the conditional probability of class 1:

$$P(y = 1 \mid x) = \sigma(w^\top x + b), \quad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

Interpretation:

- Output is a **probability** in $(0, 1)$.
- Labels are modeled as:

$$y \mid x \sim \text{Bernoulli}(a), \quad a = \sigma(w^\top x + b).$$

Likelihood and Cross-Entropy (Where the Loss Comes From)

Given data $\{(x_i, y_i)\}_{i=1}^m$ with $y_i \in \{0, 1\}$ and $a_i = \sigma(w^\top x_i + b)$:

$$P(y_i \mid x_i; w, b) = a_i^{y_i} (1 - a_i)^{1-y_i}.$$

Log-likelihood:

$$\log L(w, b) = \sum_{i=1}^m [y_i \log a_i + (1 - y_i) \log(1 - a_i)].$$

Training: maximize $\log L \Leftrightarrow$ minimize

$$\mathcal{L}(w, b) = -\frac{1}{m} \sum_{i=1}^m [y_i \log a_i + (1 - y_i) \log(1 - a_i)]$$

which is exactly the **binary cross-entropy**.

Why Logistic Regression Fits Deep Learning

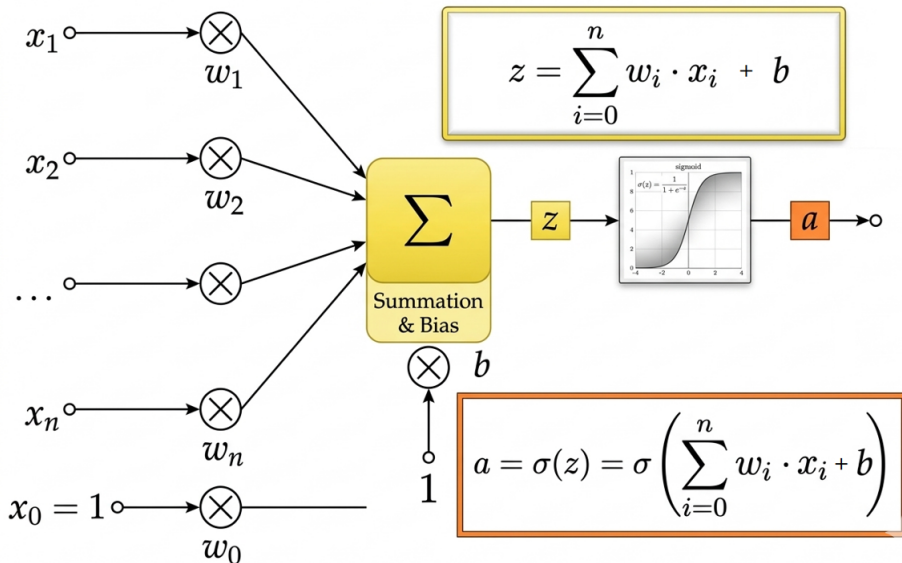
Two crucial ingredients for modern neural networks:

- **Differentiability:** $\sigma(\cdot)$ is differentiable, so gradients exist.
- **A smooth objective:** cross-entropy is differentiable and works well with gradient-based optimization.

This is the conceptual bridge from classical ML to deep learning:

Differentiable model + Differentiable loss $\Rightarrow$ Gradient-based training.

Sigmoid Neuron



Gradient Descent (For Logistic Regression)

Objective: minimize $\mathcal{L}(w, b)$.

$$w \leftarrow w - \eta \nabla_w \mathcal{L}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}.$$

Remark:

- This is standard for logistic regression because the model and loss are differentiable.
- This is **not** the historical perceptron learning rule.

Gradient Result (Logistic Regression)

Let $A = \sigma(\mathbf{X}^T \cdot \mathbf{w} + \mathbf{b})$ (vector form), $y \in \mathbb{R}^{m \times 1}$. Then:

$$\nabla_w \mathcal{L} = \frac{1}{m} X^\top (A - y), \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \mathbf{1}^\top (A - y).$$

Interpretation:

- The term $(A - y)$ is the **prediction error** (in probability space).
- This pattern will reappear in backpropagation for multi-layer networks.
- In **Appendix A** you will find the complete derivation of this result.

Subsection 4

Python Implementation

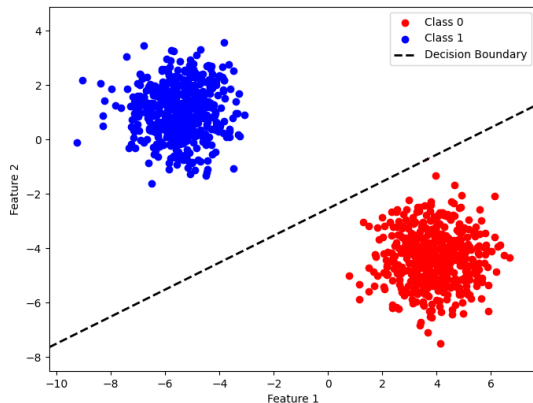
Artificial Neuron Setup

Consider a simple artificial neuron for a 2D problem:

- Inputs: x_1, x_2
- Weights: w_1, w_2
- Bias: b
- Activation function: sigmoid (σ)

The neuron's output is given by:

$$\hat{y} = \sigma(z) = \sigma(w_1x_1 + w_2x_2 + b)$$



Matrix Formulation

Logistic Neuron Implementation

- Input matrix $X \in \mathbb{R}^{m \times n}$:

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} \end{bmatrix}, \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times 1}$$

- Weights vector: $w \in \mathbb{R}^{2 \times 1}$, bias $b \in \mathbb{R}$

$$W = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \in \mathbb{R}^{2 \times 1}$$

Matrix Formulation

Logistic Neuron Implementation

Define matrix $Z \in \mathbb{R}^{m \times 1}$ explicitly:

$$\begin{aligned} Z = X \cdot W + b &= \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} + \begin{bmatrix} b \\ b \\ \vdots \\ b \end{bmatrix} \\ &= \begin{bmatrix} w_1 x_1^{(1)} + w_2 x_2^{(1)} + b \\ w_1 x_1^{(2)} + w_2 x_2^{(2)} + b \\ \vdots \\ w_1 x_1^{(m)} + w_2 x_2^{(m)} + b \end{bmatrix} \end{aligned}$$

Dimensions: $(m, 2) \cdot (2, 1) + (m, 1) = (m, 1)$

Matrix Formulation

Logistic Neuron Implementation

Explicit vectorized gradient computation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{1}{m} \begin{bmatrix} (a^{(1)} - y^{(1)})x_1^{(1)} + \dots + (a^{(m)} - y^{(m)})x_1^{(m)} \\ (a^{(1)} - y^{(1)})x_2^{(1)} + \dots + (a^{(m)} - y^{(m)})x_2^{(m)} \end{bmatrix}$$

can be compactly written as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{1}{m} \underbrace{\begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(m)} \end{bmatrix}}_{(2,m)} \left(\underbrace{\begin{bmatrix} a^{(1)} \\ a^{(2)} \\ \vdots \\ a^{(m)} \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}}_{(m,1)} \right)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{1}{m} \mathbf{X}^T \cdot (\mathbf{A} - \mathbf{y})$$

Matrix Formulation

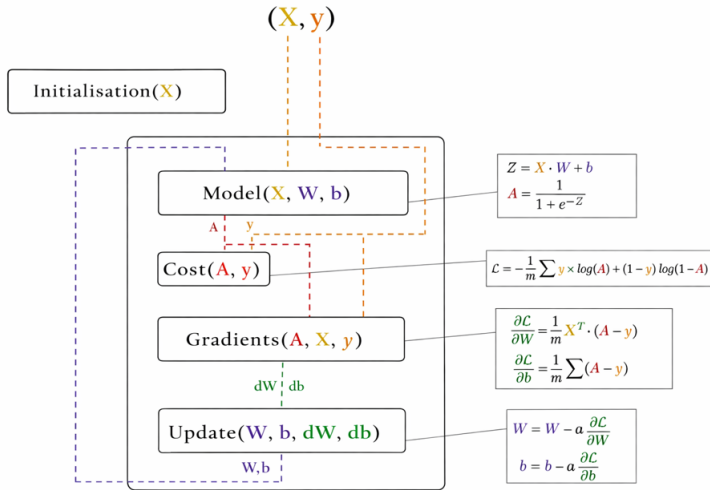
Logistic Neuron Implementation

We can easily generalize to n features:

$$\begin{aligned} Z = X \cdot W + b &= \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} + \begin{bmatrix} b \\ b \\ \vdots \\ b \end{bmatrix} \\ &= \begin{bmatrix} w_1 x_1^{(1)} + w_2 x_2^{(1)} + \dots + w_n x_n^{(1)} + b \\ w_1 x_1^{(2)} + w_2 x_2^{(2)} + \dots + w_n x_n^{(2)} + b \\ \vdots \\ w_1 x_1^{(m)} + w_2 x_2^{(m)} + \dots + w_n x_n^{(m)} + b \end{bmatrix} \end{aligned}$$

Gradient of Logistic Loss – Vectorized Form

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}} &= \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial w_n} \end{bmatrix} = \begin{bmatrix} \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) x_1^{(i)} \\ \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) x_2^{(i)} \\ \vdots \\ \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) x_n^{(i)} \end{bmatrix} \\ &= \frac{1}{m} \underbrace{\begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(m)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(m)} \end{bmatrix}}_{(n, m)} \cdot \underbrace{\left(\begin{bmatrix} a^{(1)} \\ a^{(2)} \\ \vdots \\ a^{(m)} \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \right)}_{(m, 1)}\end{aligned}$$



Python Code - Initialization

Logistic Neuron: Implementation

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \in \mathbb{R}^{n \times 1}, \quad b \in \mathbb{R}$$

```
def initialisation(X):  
    W = np.random.randn(X.shape[1], 1)  
    b = np.random.randn(1)  
    return (W, b)
```

Python Code - Model

Logistic Neuron: Implementation

$$\mathbf{Z} = \mathbf{X} \cdot \mathbf{W} + b, \quad \mathbf{A} = \frac{1}{1 + e^{-\mathbf{Z}}}$$

```
def model(X, W, b):  
    Z = X.dot(W) + b  
    A = 1 / (1 + np.exp(-Z))  
    return A
```

Python Code - Loss Function

Logistic Neuron: Implementation

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)}) \right]$$

```
def log_loss(A, y):  
    return 1 / len(y) * np.sum(-y * np.log(A) - (1 - y) * np.log(1  
        - A))
```


Python Code - Gradient and Update

$$W = W - \alpha \frac{\partial \mathcal{L}}{\partial W}, \quad b = b - \alpha \frac{\partial \mathcal{L}}{\partial b}$$

```
def gradients(A, X, y):  
    dW = 1 / len(y) * np.dot(X.T, A - y)  
    db = 1 / len(y) * np.sum(A - y)  
    return (dW, db)  
  
def update(dW, db, W, b, learning_rate):  
    W = W - learning_rate * dW  
    b = b - learning_rate * db  
    return (W, b)
```

Python Code - Predict

Logistic Neuron: Implementation

$$A = \sigma(X \cdot W + b)$$

```
def predict(X, W, b):  
    A = model(X, W, b)  
    # print(A)  
    return A >= 0.5
```

Python Code

Logistic Neuron: Implementation

```
def artificial_neuron(X, y,
                     learning_rate = 0.1,
                     n_iter = 250):
    W, b = initialisation(X)
    Loss = []
    history = []
    for i in range(n_iter):
        A = model(X, W, b)
        Loss.append(log_loss(A, y))
        dW, db = gradients(A, X, y)
        W, b = update(dW, db, W, b, learning_rate)
        history.append([W, b, Loss, i])
    y_pred = predict(X, W, b)
    print("Accuracy Score: ", accuracy_score(y, y_pred))
    plt.plot(Loss)
    plt.show()
    return (W, b, history)
```

Summary: What We Will Build Upon

Perceptron (historical):

- hard decision $\mathbb{I}(w^\top x + b \geq 0)$
- mistake-driven updates

Logistic regression (modern baseline for deep learning):

- probabilistic output $\sigma(w^\top x + b)$
- Bernoulli likelihood $\Rightarrow$ cross-entropy
- gradient descent optimization

Subsection 5

Non Linear Separable Sets Limits of the Single Neuron

Motivation: Where We Left Off

We showed that **logistic regression** solves the optimization problem with a differentiable model and a smooth loss:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

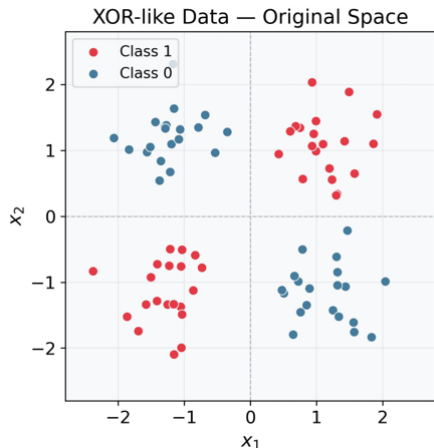
But it is still a linear classifier.

Its decision boundary is always a hyperplane — no matter how we tune $\mathbf{w}$ and b .

The question

What happens when the data **cannot** be separated by a hyperplane?

The XOR Problem



Problem definition.

Class assignment depends on a *nonlinear* rule:

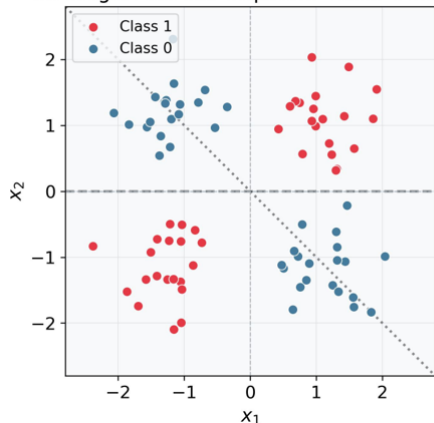
$$y = 1 \text{ iff } x_1 x_2 > 0$$

i.e. points in **Quadrants I and III** belong to **Class 1**, the rest to **Class 0**.

This is the classic **XOR-like** pattern: the label depends on the *product* (interaction) of the two features, not on each feature alone.

Why a Single Neuron Fails

No Single Line Can Separate These Classes



No matter which line we try:

$$w_1x_1 + w_2x_2 + b = 0$$

it **always** misclassifies a substantial fraction of the data.

The problem is not optimization.

A logistic neuron can find the best possible line — but no line is good enough.

The problem is the representation: the raw features (x_1, x_2) do not expose the structure needed for a linear decision.

The Core Idea: Learn a New Coordinate System

Key insight (Goodfellow et al., Ch. 6)

Instead of classifying in the *original* space, transform the input into a **new space** where the problem *becomes* linearly separable.

Introduce a **hidden layer** $\mathbf{h}(\mathbf{x})$ with two neurons:

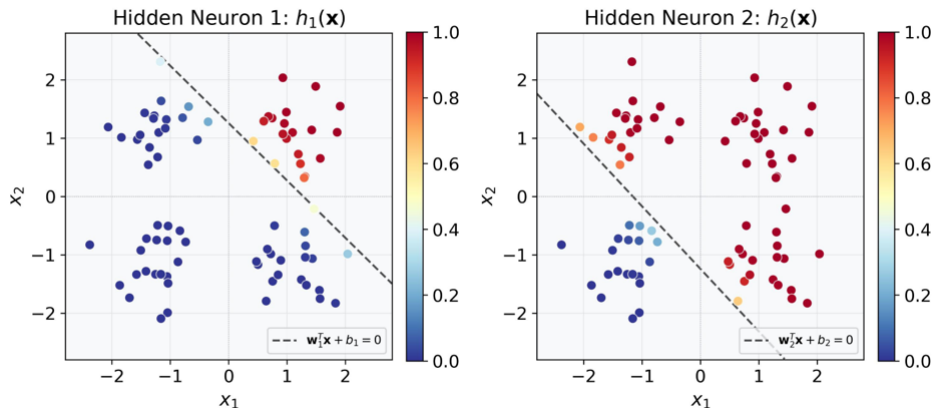
$$h_j(\mathbf{x}) = \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j), \quad j = 1, 2$$

Then classify in the *transformed* space:

$$\mathbf{x} \xrightarrow{\mathbf{h}(\mathbf{x})} (h_1, h_2) \xrightarrow{\text{linear}} \hat{y}$$

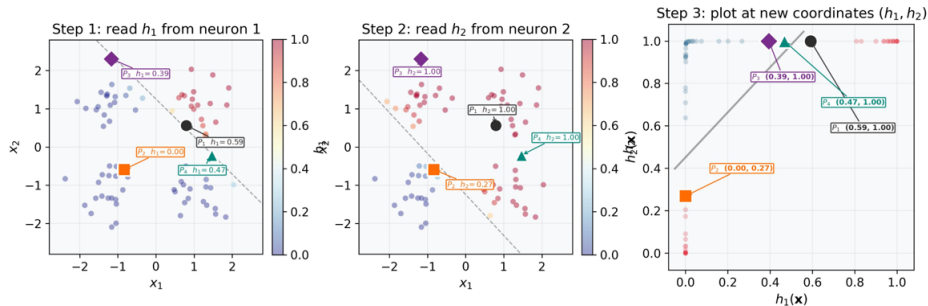
The hidden layer does not add complexity for its own sake.
It *builds a coordinate system* in which a hyperplane suffices.

What Each Hidden Neuron Computes



Each neuron partitions the original space with its own *linear boundary* and then “squashes” the result through σ . Individually, neither h_1 nor h_2 separates the classes — but **together** they create a representation where a linear classifier succeeds.

Tracking Points: From (x_1, x_2) to (h_1, h_2)

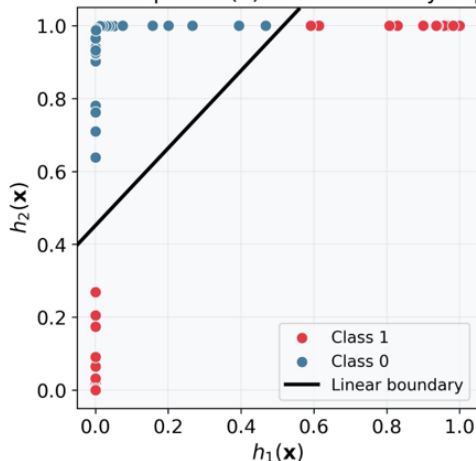


	(x_1, x_2)	Quadrant	h_1	h_2	New coords
P_1	(+0.79, +0.56)	I (cls 1)	0.59	1.00	(0.59, 1.00)
P_2	(-0.83, -0.59)	III (cls 1)	0.00	0.27	(0.00, 0.27)
P_3	(-1.17, +2.31)	II (cls 0)	0.39	1.00	(0.39, 1.00)
P_4	(+1.46, -0.21)	IV (cls 0)	0.47	1.00	(0.47, 1.00)

Recipe: read h_1 from the left panel, h_2 from the center. The pair (h_1, h_2) is the new address in the right panel.

The Transformed Space Is Linearly Separable

Transformed Space $\mathbf{h}(\mathbf{x})$ — Now Linearly Separable!



After the hidden layer, each point $\mathbf{x}$ is mapped to coordinates $(h_1, h_2) \in (0, 1)^2$.

In this new space the two classes occupy **separable clusters**.

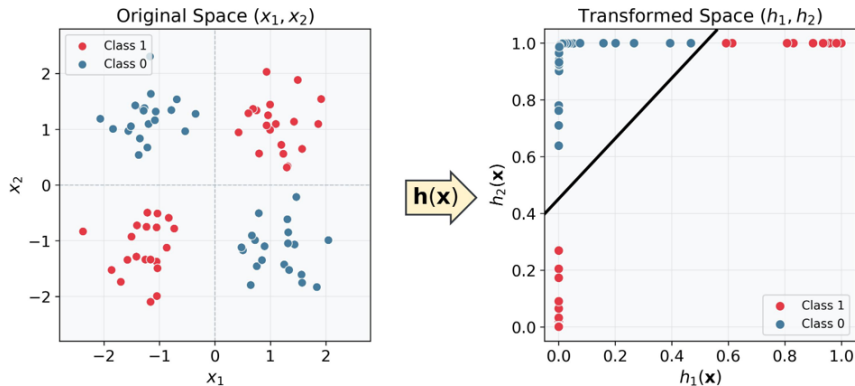
A single linear boundary

$$w'_1 h_1 + w'_2 h_2 + b' = 0$$

now suffices (black line).

This is exactly what the output neuron does: it is a logistic regression in $\mathbf{h}$ -space.

Side by Side: Original $\rightarrow$ Transformed



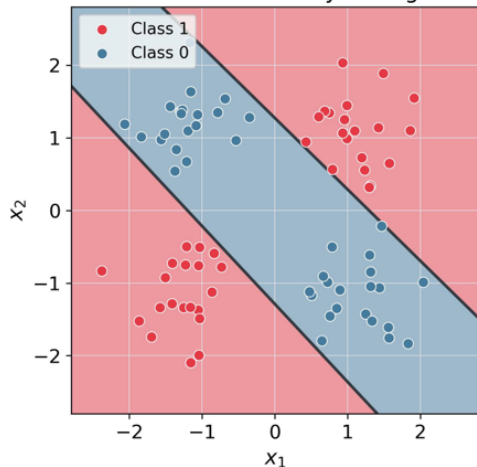
Left: no linear boundary works.

The hidden layer has *learned* the feature interaction that we would have had to engineer by hand (e.g. $x_1 x_2$).

Right: after $h(\mathbf{x})$, a straight line is all we need.

The Decision Boundary Back in the Original Space

Nonlinear Decision Boundary in Original Space



When we map the linear boundary from **h -space back** to (x_1, x_2) -space, it becomes a **nonlinear curve**.

The network has effectively learned:

- 1 A nonlinear feature map **$h(x)$**
- 2 A linear classifier in that space

Together, these two steps produce a complex decision boundary — while each step alone is simple.

This is the fundamental architecture of *every* feedforward neural network.

Learning a Representation: Resume

Instead of learning $x \rightarrow y$ directly, we change the objective:

$$x \rightarrow h(x) \rightarrow y$$

We transform the input $h(x) = \sigma(W^{[1]}x + b^{[1]})$, then classify:

$$P(y = 1|x) = \sigma(w^T h(x) + b)$$

This is the core idea of Deep Learning:

- We are not fitting a curve — we are building a **coordinate system where the problem becomes linear**
- The hidden layer replaces manual feature design

Classical ML: feature engineering $\rightarrow$ model

Deep Learning: model $\rightarrow$ features $\rightarrow$ prediction

Neural Networks

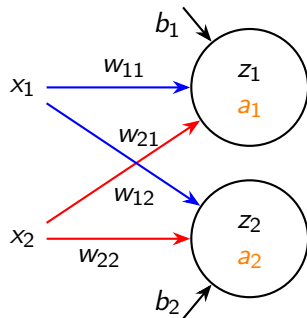
Subsection 1

Architecture

Nodes and Layers

Architecture

The first step is to add some node making a **layer**

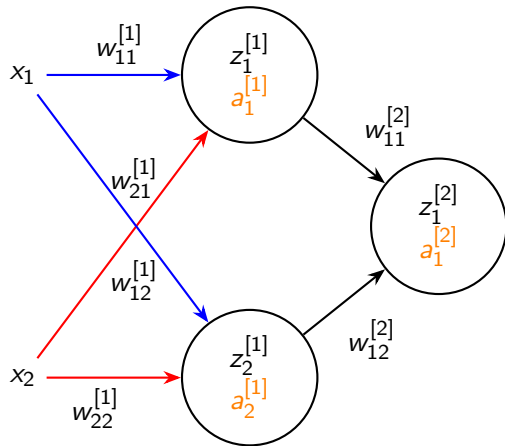


$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$
$$a_1 = \frac{1}{1 + e^{-z_1}}$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$
$$a_2 = \frac{1}{1 + e^{-z_2}}$$

Nodes and Layers

Architecture



$$z_1^{[1]} = w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + b_1^{[1]}$$
$$a_1^{[1]} = \frac{1}{1 + e^{-z_1^{[1]}}}$$

$$z_1^{[2]} = w_{11}^{[2]}a_1^{[1]} + w_{12}^{[2]}a_2^{[1]} + b_1^{[2]}$$
$$a_1^{[2]} = \frac{1}{1 + e^{-z_1^{[2]}}}$$

$$z_2^{[1]} = w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + b_2^{[1]}$$
$$a_2^{[1]} = \frac{1}{1 + e^{-z_2^{[1]}}}$$

What the Network Is Actually Doing

The network performs a sequence of computations:

$$x \rightarrow z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]}$$

Each neuron only does two things:

1) Linear combination $z = w^T x + b$

2) Nonlinear transformation $a = \sigma(z)$

A neural network is just the repeated application of these two steps.

Nodes and Layers

Architecture

As we know it's convenient to write in a more compact form the computation using the matrix notation:

$$\begin{pmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \\ w_{14} & w_{24} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix} \Rightarrow Z^{[1]} = W^{[1]} \cdot X + B^{[1]} \quad (1)$$

$$A^{[1]} = \sigma \left(Z^{[1]} \right)$$

Nodes and Layers

Architecture

The result of the hidden layer is applied to the output node which will perform another linear operation with a different set of weights, $W^{[2]}$:

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + B^{[2]}$$

and the final output will be the result of the application of the output node activation function to this value:

$$\hat{y} = A^{[2]} = \sigma \left(Z^{[2]} \right)$$

Subsection 2

The Network as a Mathematical Function

What Are We Differentiating?

Training a neural network means solving an optimization problem:

$$\min_{\theta} L(\theta)$$

But what exactly is the function $L(\theta)$?

Before computing derivatives, we must write the model as a single mathematical function.

The Network is a Function Composition

The neural network is not a collection of equations.
It is a composition of functions:

$$X \xrightarrow{f_1} Z^{[1]} \xrightarrow{\sigma} A^{[1]} \xrightarrow{f_2} Z^{[2]} \xrightarrow{\sigma} A^{[2]}$$

Each block is a function applied to the output of the previous one.
Backpropagation will simply differentiate this composition.

Explicit Functional Form of the Network

For a two-layer neural network:

$$A^{[1]}(X) = \sigma(W^{[1]}X + b^{[1]})$$

$$A^{[2]}(X) = \sigma\left(W^{[2]}A^{[1]}(X) + b^{[2]}\right)$$

Combining:

$$A^{[2]}(X; \theta) = \sigma\left(W^{[2]}\sigma(W^{[1]}X + b^{[1]}) + b^{[2]}\right)$$

This is a single function parameterized by θ .

The Loss Function

We train the network using the cross-entropy loss:

$$L(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log A_i^{[2]} + (1 - y_i) \log(1 - A_i^{[2]}) \right]$$

The training problem is therefore:

$$\min_{\theta} L(\theta)$$

All backpropagation does is compute the gradient:

$$\nabla_{\theta} L(\theta)$$

Subsection 3

Backpropagation

Backpropagation for Logistic Regression

Recall the result for a **single neuron**:

$$Z = W \cdot X + b \quad \text{linear model}$$

$$A = \frac{1}{1 + e^{-Z}} \quad \text{activation function}$$

$$\mathcal{L} = -\frac{1}{m} \sum [y \cdot \log(A) + (1 - y) \cdot \log(1 - A)] \quad \text{loss function}$$

Gradient equations:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial A} \cdot \frac{\partial A}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial A} \cdot \frac{\partial A}{\partial Z} \cdot \frac{\partial Z}{\partial b}$$

Backpropagation for a 2-Layer Neural Network

Now we have to calculate the gradient of the loss function with respect to each set of weights, one for each layer.

$$\begin{aligned} A^{[2]} &= \sigma \left(Z^{[2]} \right) = \sigma \left(W^{[2]} \cdot A^{[1]} + B^{[2]} \right) \\ &= \sigma \left(W^{[2]} \cdot \sigma \left(Z^{[1]} \right) + B^{[2]} \right) \\ &= \sigma \left[W^{[2]} \cdot \sigma \left(W^{[1]} \cdot X + B^{[1]} \right) + B^{[2]} \right] \end{aligned}$$

As you can see $A^{[2]}$ depends on both $W^{[1]}$ and $W^{[2]}$. The specification of what a layer does to its input data is stored in the layer's weights. Remember once again that **learning means finding a set of values for the weights of all layers in a network, such that the network will correctly map example inputs to their associated targets**

Backpropagation for a 2-Layer Neural Network - 1

Gradient Computation for Layer 2

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}}$$

Backpropagation for a 2-Layer Neural Network - 2

Gradient Computation for Layer 2

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

Backpropagation for a 2-Layer Neural Network - 3

Gradient Computation for Layer 2

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

Backpropagation for a 2-Layer Neural Network - 4

Gradient Computation for **Layer 1**

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}}$$

Backpropagation for a 2-Layer Neural Network - 5

Gradient Computation for **Layer 1**

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

Backpropagation for a 2-Layer Neural Network - 6

Gradient Computation for **Layer 1**

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}}$$

Backpropagation for a 2-Layer Neural Network - 7

Gradient Computation for **Layer 1**

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}}$$

Backpropagation for a 2-Layer Neural Network - 8

Gradient Computation for **Layer 1**

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Let's simplify the gradient equations

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Backpropagation for a 2-Layer Neural Network - 10

Let's simplify the gradient equations

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Backpropagation for a 2-Layer Neural Network - 11

Let's simplify the gradient equations

$$dZ2 = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Backpropagation for a 2-Layer Neural Network - 12

Let's simplify the gradient equations

$$dZ2 = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Backpropagation for a 2-Layer Neural Network - 13

Let's simplify the gradient equations

$$dZ2 = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$dZ1 = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = dZ1 \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = dZ1 \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Backpropagation for a 2-Layer Neural Network - 14

Let's simplify the gradient equations

$$dZ2 = \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

$$dZ1 = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = dZ1 \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = dZ1 \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Backpropagation Through Two Layers

Computation of dZ2

- Given the loss function L we defined above, we can easily compute the first partial derivative with respect to $\hat{y}$:

$$\begin{aligned}\mathcal{L}(y, A^{[2]}) &= - \left[y \log A^{[2]} + (1 - y) \log (1 - A^{[2]}) \right] \\ &\Downarrow \\ \frac{\partial \mathcal{L}}{\partial A^{[2]}} &= -\frac{y}{A^{[2]}} + \frac{1 - y}{1 - A^{[2]}} = \frac{\mathbf{A}^{[2]} - \mathbf{y}}{A^{[2]} \cdot (1 - A^{[2]})}\end{aligned}$$

- Note that the numerator $\mathbf{A}^{[2]} - \mathbf{y}$ is exactly the **prediction error** of the network!

Backpropagation Through Two Layers

Computation of dZ2

$$\begin{aligned}\frac{\partial A^{[2]}}{\partial Z^{[2]}} &= \frac{\partial \sigma(Z^{[2]})}{\partial Z^{[2]}} \\ &= \sigma(Z^{[2]}) \cdot [1 - \sigma(Z^{[2]})] \\ &= A^{[2]} (1 - A^{[2]})\end{aligned}$$

$$\begin{aligned}dZ2 &= \frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}} \\ &= \frac{A^{[2]} - y}{A^{[2]} \cdot (1 - A^{[2]})} \cdot A^{[2]} (1 - A^{[2]}) \\ &= A^{[2]} - y\end{aligned}$$

Backpropagation Through Two Layers

Computation of 2nd-Layer Gradients

Remembering that

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

We have

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W^{[2]}} &= dZ2 \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}} = dZ2 \cdot A^{[1]} \\ &= (A^{[2]} - y) \cdot A^{[1]}\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial b^{[2]}} &= dZ2 \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}} = dZ2 \\ &= (A^{[2]} - y)\end{aligned}$$

Backpropagation Through Two Layers

Computation of $dZ1$

$$dZ1 = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}}$$

From

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

We have immediately

$$\frac{\partial Z^{[2]}}{\partial A^{[1]}} = W^{[2]}$$

$$\frac{\partial A^{[1]}}{\partial Z^{[1]}} = \sigma(Z^{[1]}) [1 - \sigma(Z^{[1]})] = A^{[1]} (1 - A^{[1]})$$

$$\Rightarrow dZ1 = (A^{[2]} - y) \cdot W^{[2]} \cdot A^{[1]} (1 - A^{[1]})$$

Backpropagation Through Two Layers

Computation of 1st-Layer Gradients

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W^{[1]}} &= (A^{[2]} - y) \cdot W^{[2]} \cdot A^{[1]} (1 - A^{[1]}) \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}} \\ &= (A^{[2]} - y) \cdot W^{[2]} \cdot A^{[1]} (1 - A^{[1]}) \cdot X\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial b^{[1]}} &= (A^{[2]} - y) \cdot W^{[2]} \cdot A^{[1]} (1 - A^{[1]}) \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}} \\ &= (A^{[2]} - y) \cdot W^{[2]} \cdot A^{[1]} (1 - A^{[1]})\end{aligned}$$

Backpropagation Through Two Layers

Gradient equations for the 2-layer neural network (in red the 2nd layer, in violet the 1st):

$$dZ2 = (A^{[2]} - y)$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = dZ2 \cdot A^{[1]}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ2$$

$$dZ1 = (A^{[2]} - y) \cdot W^{[2]} \cdot A^{[1]} (1 - A^{[1]})$$

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = dZ1 \cdot X$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = dZ1$$

The Computational Graph Behind Backpropagation

All derivatives we are about to compute follow the same path:

$$W^{[1]} \rightarrow Z^{[1]} \rightarrow A^{[1]} \rightarrow Z^{[2]} \rightarrow A^{[2]} \rightarrow L$$

Backpropagation simply applies the chain rule along this graph:

$$\frac{\partial L}{\partial W^{[1]}} = \frac{\partial L}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial A^{[1]}} \frac{\partial A^{[1]}}{\partial Z^{[1]}} \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

We are not computing many formulas.

We are repeating the same rule along a path.

Applying the Chain Rule: Layer 2

Recall the full model:

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}, \quad A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}, \quad A^{[2]} = \sigma(Z^{[2]})$$

$$\mathcal{L} = -\frac{1}{m} \sum \left[y \cdot \log(A^{[2]}) + (1 - y) \cdot \log(1 - A^{[2]}) \right]$$

For the **output layer**, the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \underbrace{\frac{\partial \mathcal{L}}{\partial A^{[2]}} \cdot \frac{\partial A^{[2]}}{\partial Z^{[2]}}}_{dZ2} \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = dZ2 \cdot \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

Applying the Chain Rule: Layer 1

For the **hidden layer**, the chain extends further back through the graph:

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \underbrace{dZ^{[2]} \cdot \frac{\partial Z^{[2]}}{\partial A^{[1]}} \cdot \frac{\partial A^{[1]}}{\partial Z^{[1]}}}_{dZ^{[1]}} \cdot \frac{\partial Z^{[1]}}{\partial W^{[1]}}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = dZ^{[1]} \cdot \frac{\partial Z^{[1]}}{\partial b^{[1]}}$$

Observe the recursive pattern:

- Each layer computes a *local error signal* $dZ^{[\ell]}$ from the layer above
- The gradients for weights and biases follow the same form at every layer
- This is the algorithm: **propagate dZ backward, compute gradients locally**

Backpropagation Through Two Layers

Complete Gradient Descent Update — Putting It All Together

Step 1: Compute Gradients (backward pass)

Output Layer ($\ell = 2$):

$$dZ^{[2]} = A^{[2]} - y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} \cdot A^{[1]\top}$$

$$db^{[2]} = \frac{1}{m} \sum dZ^{[2]}$$

Hidden Layer ($\ell = 1$):

$$dZ^{[1]} = W^{[2]\top} \cdot dZ^{[2]} \odot A^{[1]}(1 - A^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} \cdot X^\top$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$

Step 2: Update Parameters

Output Layer ($\ell = 2$):

$$W^{[2]} \leftarrow W^{[2]} - \eta \cdot dW^{[2]}$$

$$b^{[2]} \leftarrow b^{[2]} - \eta \cdot db^{[2]}$$

Hidden Layer ($\ell = 1$):

$$W^{[1]} \leftarrow W^{[1]} - \eta \cdot dW^{[1]}$$

$$b^{[1]} \leftarrow b^{[1]} - \eta \cdot db^{[1]}$$

η = learning rate (hyperparameter)

Repeat until convergence or early stopping.

Backpropagation Through Two Layers

Explicit Update Equations — Fully Expanded Form

Substituting the gradient expressions into $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$:

Output Layer ($\ell = 2$):

$$\mathbf{W}^{[2]} \leftarrow \mathbf{W}^{[2]} - \frac{\eta}{m} \cdot (\mathbf{A}^{[2]} - \mathbf{y}) \cdot \mathbf{A}^{[1]\top} \quad (2)$$

$$\mathbf{b}^{[2]} \leftarrow \mathbf{b}^{[2]} - \frac{\eta}{m} \cdot \sum (\mathbf{A}^{[2]} - \mathbf{y}) \quad (3)$$

Hidden Layer ($\ell = 1$):

$$\mathbf{W}^{[1]} \leftarrow \mathbf{W}^{[1]} - \frac{\eta}{m} \cdot \underbrace{\left[\mathbf{W}^{[2]\top} \cdot (\mathbf{A}^{[2]} - \mathbf{y}) \right]}_{\text{error backpropagated}} \odot \underbrace{\mathbf{A}^{[1]}(1 - \mathbf{A}^{[1]})}_{\text{local } \sigma'} \cdot \mathbf{X}^{\top} \quad (4)$$

$$\mathbf{b}^{[1]} \leftarrow \mathbf{b}^{[1]} - \frac{\eta}{m} \cdot \sum \left[\mathbf{W}^{[2]\top} \cdot (\mathbf{A}^{[2]} - \mathbf{y}) \odot \mathbf{A}^{[1]}(1 - \mathbf{A}^{[1]}) \right] \quad (5)$$

Backpropagation Through Two Layers

Explicit Update Equations — Fully Expanded Form

Key: The prediction error $(A^{[2]} - y)$ propagates through *all* update equations. $\odot$ = element-wise (Hadamard) product. Eqs. (??)–(??) correspond to the Python functions `back_propagation` + `update`.

Subsection 4

Python Implementation

Weights Initialization

- Our first neural network has 1 hidden layer and 2 layers in total (hidden layer + output layer), so there are 4 weight matrices to initialize ($W^{[1]}, b^{[1]}$ and $W^{[2]}, b^{[2]}$).
- n_0 : Number of neurons in the input layer; n_1 : Number of neurons in the hidden layer; n_2 : Number of neurons in the output layer.

```
def initialisation(n0, n1, n2):  
    W1 = np.random.randn(n1, n0)  
    b1 = np.random.randn(n1, 1)  
    W2 = np.random.randn(n2, n1)  
    b2 = np.random.randn(n2, 1)  
    # Store all parameters in a dictionary for convenience  
    parametres = {  
        'W1': W1, # Weights from input to hidden  
        'b1': b1, # Biases for hidden layer  
        'W2': W2, # Weights from hidden to output  
        'b2': b2 # Biases for output layer  
    }  
    return parametres
```

Forward Propagation

$$\begin{aligned} A^{[2]} &= \sigma \left(Z^{[2]} \right) = \sigma \left(W^{[2]} \cdot A^{[1]} + B^{[2]} \right) = \sigma \left(W^{[2]} \cdot \sigma \left(Z^{[1]} \right) + B^{[2]} \right) \\ &= \sigma \left[W^{[2]} \cdot \sigma \left(W^{[1]} \cdot X + B^{[1]} \right) + B^{[2]} \right] \end{aligned}$$

```
def forward_propagation(X, parameters):
    W1 = parameters['W1']  # Weights from input to hidden layer
    b1 = parameters['b1']  # Biases for hidden layer
    W2 = parameters['W2']  # Weights from hidden to output layer
    b2 = parameters['b2']  # Biases for output layer
    Z1 = W1.dot(X) + b1
    A1 = 1 / (1 + np.exp(-Z1))
    Z2 = W2.dot(A1) + b2
    A2 = 1 / (1 + np.exp(-Z2))
    activations = {
        'A1': A1,  # Activation from hidden layer
        'A2': A2   # Final prediction (output layer)
    }
    return activations
```

Backpropagation

```
def back_propagation(X, Y, activations, parameters):
    A1 = activations['A1']  # Hidden layer activation
    A2 = activations['A2']  # Output layer activation (predicted probabilities)
    W2 = parameters['W2']  # Weights from hidden to output layer
    m = Y.shape[1]
    # ----- Backpropagation for Output Layer -----
    dz2 = A2 - Y
    dW2 = (1 / m) * dz2.dot(A1.T)
    db2 = (1 / m) * np.sum(dz2, axis=1, keepdims=True)
    # ----- Backpropagation for Hidden Layer -----
    dz1 = W2.T.dot(dz2) * A1 * (1 - A1)
    dW1 = (1 / m) * dz1.dot(X.T)
    db1 = (1 / m) * np.sum(dz1, axis=1, keepdims=True)
    gradients = {
        'dW1': dW1,
        'db1': db1,
        'dW2': dW2,
        'db2': db2
    }
    return gradients
```

Weights Update

```
def update(gradients, parameters, learning_rate):
    W1 = parameters['W1']
    b1 = parameters['b1']
    W2 = parameters['W2']
    b2 = parameters['b2']
    dW1 = gradients['dW1']
    db1 = gradients['db1']
    dW2 = gradients['dW2']
    db2 = gradients['db2']
    W1 = W1 - learning_rate * dW1    # Update weights of layer 1
    b1 = b1 - learning_rate * db1    # Update biases of layer 1
    W2 = W2 - learning_rate * dW2    # Update weights of layer 2
    b2 = b2 - learning_rate * db2    # Update biases of layer 2
    parameters = {
        'W1': W1,
        'b1': b1,
        'W2': W2,
        'b2': b2
    }
    return parameters
```

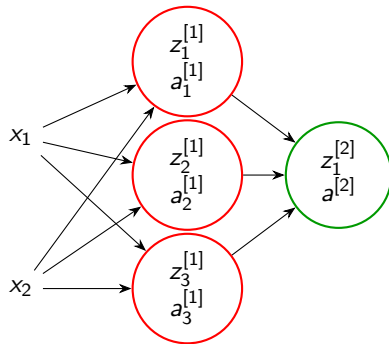
Training Loop

```
for i in range(n_iter):
    activations = forward_propagation(X_train, parameters)
    gradients = back_propagation(X_train, y_train, activations, parameters)
    parameters = update(gradients, parameters, learning_rate)
    if i % 10 == 0:
        loss = log_loss(y_train.flatten(), activations['A2'].flatten())
        train_loss.append(loss)
        y_pred = predict(X_train, parameters)
        current_accuracy = accuracy_score(y_train.flatten(), y_pred.flatten())
        train_acc.append(current_accuracy)
```

Subsection 5

From 2 Layers to C Layers — Architecture

Forward Propagation 2-Layer Network



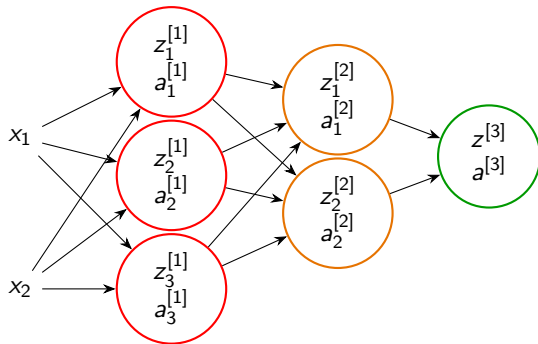
$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

Forward Propagation 3-Layer Network

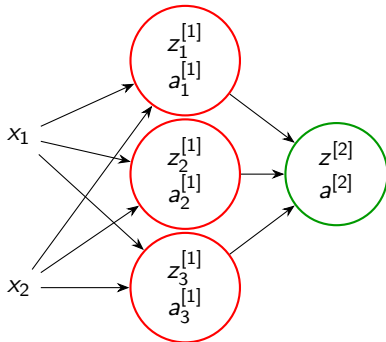


$$Z^{[1]} = W^{[1]}X + b^{[1]}$$
$$A^{[1]} = \frac{1}{1 + e^{-Z^{[1]}}}$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$
$$A^{[2]} = \frac{1}{1 + e^{-Z^{[2]}}}$$

$$Z^{[3]} = W^{[3]}A^{[2]} + b^{[3]}$$
$$A^{[3]} = \frac{1}{1 + e^{-Z^{[3]}}}$$

Backpropagation – 2-Layer Network



$$dZ^{[2]} = A^{[2]} - y$$

$$dW^{[2]} = \frac{1}{m} \cdot dZ^{[2]} \cdot A^{[1]T}$$

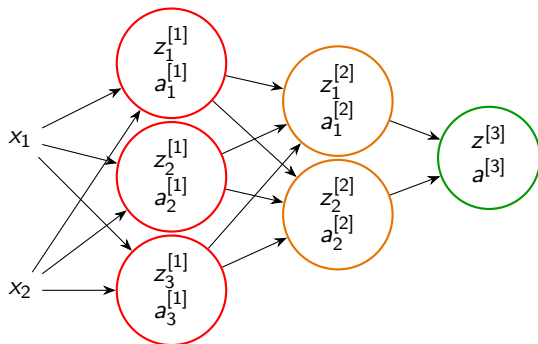
$$db^{[2]} = \frac{1}{m} \sum dZ^{[2]}$$

$$dZ^{[1]} = W^{[2]T} \cdot dZ^{[2]} \cdot A^{[1]}(1 - A^{[1]})$$

$$dW^{[1]} = \frac{1}{m} \cdot dZ^{[1]} \cdot X^T$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$

Backpropagation – 3-Layer Network



$$\begin{aligned} dZ^{[3]} &= A^{[3]} - y \\ dW^{[3]} &= \frac{1}{m} dZ^{[3]} A^{[2]T} \\ db^{[3]} &= \frac{1}{m} \sum_{\text{axes}=1} dZ^{[3]} \end{aligned}$$

$$\begin{aligned} dZ^{[2]} &= W^{[3]T} dZ^{[3]} \cdot A^{[2]}(1 - A^{[2]}) \\ dW^{[2]} &= \frac{1}{m} dZ^{[2]} A^{[1]T} \\ db^{[2]} &= \frac{1}{m} \sum_{\text{axes}=1} dZ^{[2]} \end{aligned}$$

$$\begin{aligned} dZ^{[1]} &= W^{[2]T} dZ^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \\ dW^{[1]} &= \frac{1}{m} dZ^{[1]} X^T \\ db^{[1]} &= \frac{1}{m} \sum_{\text{axes}=1} dZ^{[1]} \end{aligned}$$

Backpropagation – 3-Layer Network

$$\begin{aligned}dZ^{[3]} &= A^{[3]} - y \\dW^{[3]} &= \frac{1}{m} dZ^{[3]} A^{[2]T} \\db^{[3]} &= \frac{1}{m} \sum_{\text{axes}=1} dZ^{[3]}\end{aligned}$$

$$\begin{aligned}dZ^{[2]} &= W^{[3]T} dZ^{[3]} \cdot A^{[2]}(1 - A^{[2]}) \\dW^{[2]} &= \frac{1}{m} dZ^{[2]} A^{[1]T} \\db^{[2]} &= \frac{1}{m} \sum_{\text{axes}=1} dZ^{[2]}\end{aligned}$$

$$\begin{aligned}dZ^{[1]} &= W^{[2]T} dZ^{[2]} \cdot A^{[1]}(1 - A^{[1]}) \\dW^{[1]} &= \frac{1}{m} dZ^{[1]} X^T \\db^{[1]} &= \frac{1}{m} \sum_{\text{axes}=1} dZ^{[1]}\end{aligned}$$

$$dZ^{[c]} = A^{[c]} - y$$

$$dW^{[c]} = \frac{1}{m} \cdot dZ^{[c]} \cdot A^{[c-1]T}$$

$$db^{[c]} = \frac{1}{m} \sum_{\text{axes}=1} dZ^{[c]}$$

$$dZ^{[c-1]} = W^{[c]T} \cdot dZ^{[c]} \cdot A^{[c-1]}(1 - A^{[c-1]})$$

where c is the layer number and C the last layer

From Two Layers to a General Deep Network

Parameterizing a Network of Arbitrary Depth

The 2-layer network used **named variables**: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$. To generalize to C layers, we store all parameters in a single **dictionary** indexed by layer number $c = 1, \dots, C$:

Architecture specification

A Python list defines the full topology:

```
dimensions = [n0, n1, n2, ..., nC]
```

where n_0 = input features, n_C = output (1 for binary).

Key change: every hard-coded “W1, W2” becomes a **loop** over c . For instance, the 2-layer

$W1 = \text{random}(n1, n0); W2 = \text{random}(n2, n1)$ becomes simply

for c in $1..C$: $W\{c\} = \text{random}(n_c, n_{c-1})$.

From Two Layers to a General Deep Network

Parameterizing a Network of Arbitrary Depth

Initialization — generalized

```
def initialization(dimensions):  
    parameters = {}  
    C = len(dimensions)  
    for c in range(1, C):  
        parameters['W' + str(c)] = \  
            np.random.randn(  
                dimensions[c],  
                dimensions[c-1])  
        parameters['b' + str(c)] = \  
            np.random.randn(  
                dimensions[c], 1)  
    return parameters
```

From Two Layers to a General Deep Network

Example: dimensions = [2, 16, 32, 1]

$\Rightarrow C = 3$ layers, with:

$$W^{[1]} \in \mathbb{R}^{16 \times 2}, \quad W^{[2]} \in \mathbb{R}^{32 \times 16}, \quad W^{[3]} \in \mathbb{R}^{1 \times 32}$$

Forward and Backward Propagation — Generalized

One Loop Replaces All Layer-Specific Equations

Forward propagation Set $A^{[0]} = X$, then for $c = 1, \dots, C$:

$$Z^{[c]} = W^{[c]}A^{[c-1]} + b^{[c]}, \quad A^{[c]} = \sigma(Z^{[c]})$$

```
def forward_propagation(X, parameters):
    activations = {'A0': X}
    C = len(parameters) // 2
    for c in range(1, C + 1):
        Z = parameters['W'+str(c)].dot(
            activations['A'+str(c-1)]) \
            + parameters['b'+str(c)]
        activations['A'+str(c)] = \
            1 / (1 + np.exp(-Z))
    return activations
```


Forward and Backward Propagation — Generalized

One Loop Replaces All Layer-Specific Equations

Backpropagation

Initialize $dZ^{[C]} = A^{[C]} - y$, then for $c = C, \dots, 1$:

$$dW^{[c]} = \frac{1}{m} dZ^{[c]} A^{[c-1]\top}, \quad db^{[c]} = \frac{1}{m} \sum dZ^{[c]}$$

If $c > 1$: $dZ^{[c-1]} = W^{[c]\top} dZ^{[c]} \odot A^{[c-1]}(1 - A^{[c-1]})$

```
def back_propagation(y, activations, parameters):
    m = y.shape[1]
    C = len(parameters) // 2
    dZ = activations['A'+str(C)] - y
    gradients = {}
    for c in reversed(range(1, C+1)):
        gradients['dW'+str(c)] = \
            1/m * dZ.dot(activations['A'+str(c-1)].T)
        gradients['db'+str(c)] = \
            1/m * np.sum(dZ, axis=1, keepdims=True)
        if c > 1:
            dZ = parameters['W'+str(c)].T.dot(dZ) \
                * activations['A'+str(c-1)] \
                * (1 - activations['A'+str(c-1)])
    return gradients
```

Parameter Update and Training Loop

Completing the Generalized Training Algorithm

Update rule — generalized The update applies the **same rule at every layer**:

$$W^{[c]} \leftarrow W^{[c]} - \eta dW^{[c]}, \quad b^{[c]} \leftarrow b^{[c]} - \eta db^{[c]}$$

```
def update(gradients, parameters,
           learning_rate):
    C = len(parameters) // 2
    for c in range(1, C + 1):
        parameters['W' + str(c)] -= \
            learning_rate * \
            gradients['dW' + str(c)]
        parameters['b' + str(c)] -= \
            learning_rate * \
            gradients['db' + str(c)]
    return parameters
```

Parameter Update and Training Loop

Completing the Generalized Training Algorithm

Training loop

```
for i in range(n_iter):  
    activations = forward_propagation(  
        X_train, parameters)  
    gradients = back_propagation(  
        y_train, activations,  
        parameters)  
    parameters = update(  
        gradients, parameters,  
        learning_rate)
```

Parameter Update and Training Loop

Completing the Generalized Training Algorithm

What changed from the 2-layer version?

Component	2-layer	C-layer
Architecture	hard-coded	dimensions list
Parameters	$W1, b1, W2, b2$	dictionary
Forward pass	2 explicit eqs.	<code>for c in 1..C</code>
Backprop	2 explicit eqs.	<code>for c in C..1</code>
Update	4 assignments	<code>for c in 1..C</code>
Training loop	identical	

A Key Challenge in Deep Learning

- As networks get deeper, training becomes more difficult.
- A core problem: **gradients may vanish or explode** during backpropagation.
- This makes learning:
 - Very slow (vanishing gradients)
 - Unstable (exploding gradients)

Why it happens

In backpropagation, the gradient is computed through repeated multiplication. For deep networks, this can cause gradients to shrink or grow exponentially.

Vanishing Gradients

- Occurs when gradients become **very small** as they flow backward.
- Common when using the **sigmoid** or **tanh** activation functions.
- Leads to:
 - Neurons in early layers learning extremely slowly.
 - Very flat loss surfaces.

Mathematical Insight

Derivatives of sigmoid are always < 0.25 , and repeated multiplication in deep networks makes gradients vanish.

Exploding Gradients

- Opposite of vanishing: gradients become **very large**.
- Causes:
 - Unstable weight updates
 - Diverging loss
 - Numerical overflow (NaNs)
- More common in networks with:
 - Poor initialization
 - Large learning rates

Symptom

Training loss oscillates wildly or becomes NaN.

How to Address Gradient Issues

- **Use better activation functions:**
 - ReLU, Leaky ReLU reduce vanishing gradient problems
- **Initialize weights carefully:**
 - Xavier (Glorot) initialization
 - He initialization (for ReLU)
- **Apply normalization:**
 - Batch Normalization standardizes activations layer-by-layer
- **Use gradient clipping:**
 - Limit the norm of gradients to prevent explosion

Takeaway

Deep learning is powerful, but requires careful training design to avoid gradient instability.

Subsection 6

Activation Functions Beyond the Sigmoid

Why the Sigmoid Is Not Enough

We have used the sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ throughout.

Problems in deep networks:

- **Vanishing gradients:** $\sigma'(z) \leq 0.25$ always. In a network with L layers, gradients shrink as $\sim (0.25)^L$.
- **Not zero-centered:** outputs are in $(0, 1)$, causing zig-zag updates.
- **Saturation:** for large $|z|$, the gradient is nearly zero $\Rightarrow$ neurons stop learning.

Key insight

Modern deep learning replaces the sigmoid with activation functions that do **not** suffer from these issues.

The ReLU Activation

The **Rectified Linear Unit** (Nair & Hinton, 2010):

$$g(z) = \max(z, 0)$$

Advantages:

- Gradient is 1 for $z > 0 \Rightarrow$ no vanishing gradient
- Very fast to compute
- Induces **sparsity**: negative pre-activations are set to zero
- Empirically the most widely used activation

Derivative:

$$g'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

Limitation: “dead neurons” — if $z < 0$ for all training inputs, the gradient is permanently zero.

ReLU Variants

Leaky ReLU (Maas et al., 2013)

$$g(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases} \quad \text{with } \alpha \approx 0.01$$

Prevents dead neurons by allowing a small gradient when $z < 0$.

ELU — Exponential Linear Unit (Clevert et al., 2015)

$$g(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$$

Smooth approximation that pushes mean activations closer to zero.

Swish / SiLU (Ramachandran et al., 2017)

$$g(z) = z \cdot \sigma(z)$$

Smooth, non-monotonic. Often outperforms ReLU in very deep networks.

Choosing an Activation Function — Practical Guidelines

Activation	Range	Vanishing?	Typical use
Sigmoid $\sigma(z)$	$(0, 1)$	Yes	Output layer (binary)
Tanh	$(-1, 1)$	Yes	Recurrent networks
ReLU	$[0, \infty)$	No	Hidden layers (default)
Leaky ReLU	$(-\infty, \infty)$	No	Hidden layers
Softmax	$(0, 1)^K$	—	Output (multiclass)
Linear	$(-\infty, \infty)$	No	Output (regression)

Rule of thumb:

- **Hidden layers:** start with ReLU
- **Output layer:** match the problem (sigmoid for binary, softmax for multiclass, linear for regression)

Subsection 7

Stochastic Optimization

From Full-Batch to Stochastic Gradient Descent

So far we computed the gradient over **all** m training examples:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta) = \theta - \eta \cdot \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \ell_i(\theta)$$

Problem: when m is very large, each gradient step is extremely expensive.

Example from finance: In Gu, Kelly & Xiu (2019), the dataset has $\sim 30,000$ stocks over 720 months. Computing the full gradient at each step is prohibitive.

Key idea: Stochastic Gradient Descent (SGD)

Approximate the gradient using a small random **mini-batch** $\mathcal{B}_t \subset \{1, \dots, m\}$:

$$\theta \leftarrow \theta - \eta \cdot \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \ell_i(\theta)$$

Mini-Batch Gradient Descent

Three regimes:

Method	Batch size	Trade-off
Full-batch GD	$ \mathcal{B} = m$	Accurate but slow
Pure SGD	$ \mathcal{B} = 1$	Fast but very noisy
Mini-batch SGD	$ \mathcal{B} \in [32, 512]$	Best compromise

Training protocol (one epoch = one pass through the data):

- 1 Shuffle the training set randomly
- 2 Split into mini-batches of size b
- 3 For each mini-batch: compute gradient, update θ
- 4 Repeat for multiple epochs

Mini-batch SGD introduces **noise** in the gradient estimate, which can actually help escape local minima.

The Problem with a Fixed Learning Rate

In SGD with a constant learning rate η :

- **Too large** η : the optimizer overshoots and diverges
- **Too small** η : convergence is extremely slow
- Near the minimum, the **noise dominates** the gradient signal $\Rightarrow$ the optimizer oscillates instead of converging

Desired behavior:

- Take large steps early in training (when far from the optimum)
- Take smaller steps later (to fine-tune)
- Adapt the step size **per-parameter**, since different weights may need different learning rates

Solution: *adaptive* optimizers that adjust the learning rate automatically.

The Adam Optimizer (Kingma & Ba, 2014)

Adam (Adaptive Moment Estimation) combines two ideas:

- **Momentum:** use a running average of past gradients (1st moment)
- **Adaptive learning rate:** scale by a running average of squared gradients (2nd moment)

Update rules at iteration t :

$$g_t = \nabla_{\theta} \mathcal{L}(\theta_{t-1}) \quad (\text{gradient on mini-batch})$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{1st moment estimate})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{2nd moment estimate})$$

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad (\text{bias correction})$$

$$\hat{v}_t = v_t / (1 - \beta_2^t) \quad (\text{bias correction})$$

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \quad (\text{parameter update})$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Why Adam Works Well

Momentum (m_t):

- Smooths out noisy gradients
- Accelerates in consistent directions
- Like a ball rolling downhill with inertia

Adaptive scaling (v_t):

- Parameters with large gradients get *smaller* updates
- Parameters with small gradients get *larger* updates
- Effectively uses a different learning rate for each parameter

Comparison

SGD	One η for all
Momentum	+ smoothing
RMSPprop	+ adaptive η
Adam	All combined

In practice

Adam is the **default optimizer** in most deep learning applications, including Gu, Kelly & Xiu (2019).

Subsection 8

Regularization

The Overfitting Problem

Overfitting occurs when a model fits the **noise** in the training data, rather than the underlying signal.

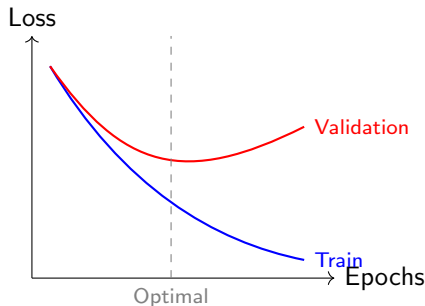
Symptoms:

- Training loss keeps decreasing
- Validation loss starts **increasing**
- Large gap between train/test performance

Why neural networks are prone:

- Many parameters (often millions)
- High model capacity
- Can memorize training data

Regularization = any technique that improves generalization by constraining model complexity.



ℓ_1 Penalization (LASSO)

Append a penalty to the loss function that discourages large weights:

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_j |\theta_j|$$

- The penalty $\lambda \sum_j |\theta_j|$ is called **LASSO** (ℓ_1) regularization
- $\lambda \geq 0$ is a hyperparameter controlling regularization strength
- **Key property:** the geometry of the ℓ_1 ball sets some weights to **exactly zero** $\Rightarrow$ **sparsity**

Effect:

- Irrelevant features are automatically discarded
- The model becomes simpler and more interpretable
- Out-of-sample performance improves by reducing noise fitting

LASSO / ℓ_1 :

$$\phi(\theta) = \lambda \sum_j |\theta_j|$$

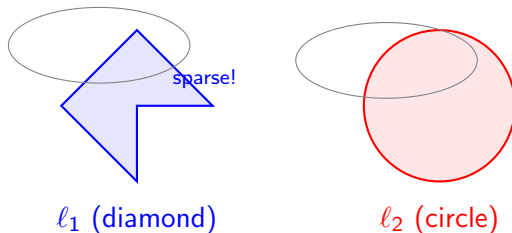
- Produces **sparse** solutions
- Feature selection built-in
- Non-differentiable at $\theta_j = 0$

Ridge / ℓ_2 :

$$\phi(\theta) = \lambda \sum_j \theta_j^2$$

- Shrinks weights toward zero but **never exactly zero**
- Differentiable everywhere
- Penalizes large weights more

ℓ_1 vs ℓ_2 Penalization



The ℓ_1 constraint set has **corners** on the axes, where some coordinates are exactly zero. The ℓ_2 constraint set is smooth $\Rightarrow$ solutions are rarely exactly sparse.

Idea: Monitor the validation loss during training and **stop before the model overfits**.

Algorithm:

- 1 Set a **patience** parameter p and initialize $j = 0$
- 2 At each training step, evaluate the validation loss $\mathcal{L}'$
- 3 If $\mathcal{L}' < \mathcal{L}_{\text{best}}^*$: update the best model, reset $j \leftarrow 0$
- 4 Otherwise: $j \leftarrow j + 1$
- 5 **Stop** when $j = p$ (no improvement for p consecutive checks)
- 6 Return the parameters θ^* corresponding to the best validation loss

Why it works:

- Training starts from small random weights (near-zero)
- Each gradient step moves θ away from zero to reduce the training loss
- Stopping early keeps θ close to zero $\Rightarrow$ equivalent to **implicit ℓ_2 regularization**

Advantage over explicit penalization

Early stopping achieves regularization at **much lower computational cost**: no need to tune λ or compute penalty gradients.

Batch Normalization (Ioffe & Szegedy, 2015)

Problem: As training progresses, the distribution of inputs to each hidden layer shifts (**internal covariate shift**). This makes training unstable, especially in deep networks.

Solution: For each mini-batch and each hidden layer, normalize activations:

$$\mu_{\mathcal{B}} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

where γ and β are **learnable** scale and shift parameters.

Batch Normalization (Ioffe & Szegedy, 2015)

Benefits:

- Stabilizes and **accelerates** training (allows higher learning rates)
- Acts as a mild regularizer
- Reduces sensitivity to weight initialization

Problem: Neural network optimization is non-convex. Different random initializations can lead to **different local optima**, each with different predictions.

Solution: Train multiple networks with different random seeds, and average their predictions:

$$\hat{y}_{\text{ensemble}} = \frac{1}{S} \sum_{s=1}^S \hat{y}^{(s)}$$

Why it works:

- Individual models make different errors
- Averaging **reduces variance** without increasing bias
- The ensemble is more **stable** and reliable than any single model

In Gu, Kelly & Xiu (2019)

They use $S = 10$ random seeds for each model configuration and average the predictions. This is their third regularization technique, alongside ℓ_1 penalization and early stopping.

Training, Validation, and Testing

The **three-way data split** is essential for reliable model evaluation:

Sample	Purpose	What is optimized
Training	Estimate model parameters θ	Weights W , biases b
Validation	Tune hyperparameters	λ , learning rate, architecture
Testing	Final, unbiased evaluation	Nothing — report results only

Critical: The test set must **never** influence any modeling decision.

For time series data (as in asset pricing), the split must respect temporal ordering:

- No future data leaks into the training set
- Expanding or rolling windows are preferred over random splits

In Gu, Kelly & Xiu (2019)

Training: 1957–1974, Validation: 1975–1986, Testing: 1987–2016 (30 years, purely out-of-sample). Models are refitted annually with an expanding training window.

Summary: The Deep Learning Toolkit

Putting it all together — the essential tools for training deep networks:

Component	Default choice	Role
Activation (hidden)	ReLU	Nonlinearity without vanishing gradients
Activation (output)	Linear / Sigmoid	Task-dependent
Loss function	MSE / Cross-entropy	Measures prediction error
Optimizer	Adam	Adaptive learning rates
ℓ_1 / ℓ_2 penalty	LASSO / Ridge	Explicit weight regularization
Early stopping	Patience-based	Implicit regularization
Batch normalization	Per-layer	Stabilizes training
Ensemble	Multiple seeds	Reduces variance

Conclusions

Conclusions

Recap: What We Learned

Part I — From the Perceptron to Backpropagation:

- The historical perceptron (step activation, mistake-driven) vs. modern differentiable models
- Multi-layer networks as **learned representations**: $x \rightarrow h(x) \rightarrow y$
- Backpropagation = systematic application of the chain rule to compute $\nabla_{\theta} \mathcal{L}$

Part II — The Modern Deep Learning Toolkit:

- **Activation functions**: ReLU $g(z) = \max(z, 0)$ solves the vanishing gradient problem
- **Optimization**: SGD on mini-batches $\rightarrow$ Adam (adaptive moments)
- **Regularization**: ℓ_1 (LASSO) for sparsity, early stopping, batch normalization, ensemble averaging
- **Evaluation**: train / validation / test split with temporal ordering for financial data

- **Factor models and PCA:**

- Linear latent factor models: $r_t = \beta f_t + u_t$
- PCA as dimension reduction for asset returns
- The equivalence between linear autoencoders and PCA

- **Conditional models and IPCA:**

- Time-varying betas: $\beta(z_{i,t-1})' = z_{i,t-1}'\Gamma$
- Why linearity is a strong and likely violated assumption
- Motivating the **conditional autoencoder** of Gu, Kelly & Xiu (2019)

References

Recommended Books

- **Deep Learning** — Ian Goodfellow, Yoshua Bengio, Aaron Courville
 - The most comprehensive and cited reference in the field
- **Neural Networks and Deep Learning** — Michael Nielsen
 - Free online book with intuitive explanations and interactive code
- **Pattern Recognition and Machine Learning** — Christopher Bishop
 - Excellent theoretical grounding in ML and probabilistic models
- **Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow** — Aurélien Géron
 - Practical, code-heavy book ideal for project-based learning

Key Papers in Neural Networks

- **Frank Rosenblatt (1958)** – *The Perceptron: A Probabilistic Model*
 - Introduced the concept of the perceptron
- **Rumelhart, Hinton and Williams (1986)** – *Learning Representations by Back-Propagating Errors*
 - Revived neural networks by introducing effective training via backpropagation
- **Kingma and Ba (2014)** – *Adam: A Method for Stochastic Optimization*
 - The adaptive optimizer used in most modern deep learning
- **Ioffe and Szegedy (2015)** – *Batch Normalization: Accelerating Deep Network Training*
 - Stabilizes training via per-layer normalization
- **Gu, Kelly and Xiu (2019)** – *Autoencoder Asset Pricing Models*
 - Uses the techniques from this lecture (ReLU, Adam, ℓ_1 , early stopping, ensembles) to build conditional factor models

- **Online Courses**

- Deep Learning Specialization – Andrew Ng (Coursera)
- Practical Deep Learning for Coders – Fast.ai

- **Blog Series / Interactive Tutorials**

- Chris Olah's Blog — deep, intuitive visualizations
- Distill.pub — high-quality explanations of ML concepts

- **Datasets and Challenges**

- UCI Machine Learning Repository
- Kaggle Datasets

Appendix A

Gradient Result for Logistic Neuron

Gradient Result

Logistic Neuron

Binary cross-entropy loss for n examples:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$$

where $a^{(i)} = \sigma(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$. We apply the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial \mathbf{w}}$$

Gradient Result

Logistic Neuron

We differentiate the loss with respect to $\mathbf{w}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = -\frac{1}{m} \sum_{i=1}^m \left[\frac{y^{(i)}}{a^{(i)}} - \frac{1 - y^{(i)}}{1 - a^{(i)}} \right] \cdot \frac{\partial a^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \mathbf{w}}$$

where:

$$z^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} + b = w_1 \cdot x_1^{(i)} + w_2 \cdot x_2^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)}) \Rightarrow \frac{\partial a^{(i)}}{\partial z^{(i)}} = a^{(i)}(1 - a^{(i)})$$

$$\frac{\partial z^{(i)}}{\partial w_j} = x_j^{(i)}$$

Gradient Result

Logistic Neuron

Combining everything:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_j} &= -\frac{1}{m} \sum_{i=1}^m \left[\frac{y^{(i)}}{a^{(i)}} - \frac{1 - y^{(i)}}{1 - a^{(i)}} \right] \cdot \frac{\partial a^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial w_j} \\ &= \frac{1}{m} \sum_{i=1}^m \left[\frac{a^{(i)} - y^{(i)}}{a^{(i)} (1 - a^{(i)})} \right] \cdot a^{(i)} (1 - a^{(i)}) \cdot x_j^{(i)} \\ &= \frac{1}{m} \sum_{i=1}^m (a^{(i)} - y^{(i)}) \cdot x_j^{(i)}\end{aligned}$$

Similarly:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (a^{(i)} - y^{(i)})$$